

Lab 3 — Architectural Justification

Restaurant Table Booking & Pre-Ordering App | Problem Statement #36

PES University — Dept. of CSE • Name: NIKHIL.P • SRN: PES1UG24CS583 • Date: 16-09-2026

Architecture Selection: We selected Microservices Architecture for the Restaurant Table Booking & Pre-Ordering System because the main services can work and scale separately.

1. Architectural style analysis

Style	Strength	Limitation	Decision
Layered	Separates presentation, business logic and data. Easy to understand.	The whole application has to be scaled together, even when only table availability has high load.	Rejected
Client–Server	Centralized control and simple data management.	If the server fails, booking, payment and kitchen operations can be affected together.	Rejected
Microservices	Services can be developed, scaled and deployed separately. Payment can be isolated.	More services mean more network communication and management work.	Selected

2. Reason one — load is mainly on table availability

NFR-001 requires table status updates to reach manager terminals and the diner floor plan within 500 ms for 200 concurrent sessions. The Table Availability Service will receive most of these updates. With microservices, this service can be scaled separately and can use a Redis cache during busy hours. Other services do not need the same amount of resources.

3. Reason two — failures should remain local

The system uses different functions such as payment, kitchen display, floor terminals and the diner app. If the payment gateway has a problem, users should still be able to view tables and the kitchen should continue its work. Separate services help keep such failures limited to the affected service instead of stopping the complete application.

4. Security advantage

NFR-002 says that raw card details should not be stored. The Payment Service can handle the payment gateway and store only payment tokens. This keeps sensitive payment information limited to one service and reduces the security scope of the application. Other services work with booking details and tokens instead of card numbers or CVV.

5. Performance benefit

The Table Availability Service can keep live table information in memory and send updates using WebSockets through the API Gateway. This reduces repeated database access and allows the service to be scaled separately. This helps the system work towards the required 500 ms p95 response time.

6. Components and interfaces modelled

Component	Source	Responsibility
Order Manager Component	Given	Handles booking, pre-order and payment flow.
Payment Service Component	Given	Connects to the payment gateway and uses tokens instead of raw card data.
Table Availability Service	Identified	Maintains table status and sends updates to terminals.
Kitchen Prep Scheduler	Identified	Calculates preparation times and sends tasks to the KDS.
Menu & Catalog Service	Identified	Provides menu items, prices and preparation times.
API Gateway	Identified	Entry point for the diner app and manager terminals; handles authentication and routing.

Interfaces shown include IBookingAPI, IFloorOpsAPI, IReservation, IMenuCatalog, IPrepSchedule, IPayment, ITableAvailability and IFloorStatusStream. The diagram also shows dependencies on the Payment Gateway, Kitchen Display System and each service's data store.